

HelixConstitution

HelixConstitution

每个项目继承的通用工程宪章——反虚假验证法则，机械化执行，以单一Git子模块共享。

概要

HelixConstitution是一套单一、与项目无关的规则手册——所有Helix/vasic-digital项目均通过Git子模块引入——它将不可妥协的工程纪律（反虚假验证、唯证据验证、数据/主机安全、文档与测试覆盖率）编码化，并传播至140多个代码库。它是整个项目家族的治理支柱，确保其整体一致性。

简要描述

一个通用、可继承的Constitution，以Git子模块形式发布。它定义了强制性、不可妥协的规则——反虚假验证门禁、假阳性免疫、数据与主机安全、覆盖率与文档纪律——每个引入项目自动继承，并可扩展但绝不削弱。

详细描述

HelixConstitution是所有通过Git子模块引入的项目共享工程实践的权威单一来源——工程法则以代码般精确的方式分发并版本锁定。其核心文件——Constitution.md——是一个约1MB、持续版本化的编号条款文档（§11.4.x 契约族，目前更新至§11.4.170），并附带针对不同代理的操作手册

（CLAUDE.md、AGENTS.md、QWEN.md、GEMINI.md），这些手册通过引用导入该文档，确保人类与所有CLI代理均遵循同一规则手册。继承机制刻意设计为三层结构：通用基础层（该子模块）、项目层（项目自身的Constitution/CLAUDE/AGENTS扩展文件）及可选的子目录层——自上而下评估，项目可强化规则，但架构上绝不允许削弱规则。其结果是一个由140多个代码库组成的家族，因共享的纪律被版本锁定而非依赖记忆，从而避免悄然偏离。

该文档坚持绝对的领域无关性：任何涉及具体供应商、硬件SKU、端口或库版本的内容，均需下放至引入项目自身的Constitution中，且普适性绝非假设——每条规则在纳入基础层前，必须通过明确的四重测试验证其合理性。其哲学核心是反虚假验证，通过一系列相互关联的契约体现——§1.1假阳性

免疫、§11.4终端用户质量契约、§11.4.6禁止猜测、§11.4.6.9实证分类法——这些契约共同构筑了一条硬性红线：发布标准绝非“测试通过”，而是“真实用户能够使用该功能”，且每个“通过”结果必须引用捕获的物理证据，否则不予认可。配套的submodules-catalogue.md（收录142个代码库）将“我们是否已有实现此功能的模块？”“转化为编写新代码前的”目录优先、扩展而非重造”本能反应。辅助脚本能从任意嵌套深度定位子模块，并将每次提交同步至四个独立Git服务商，确保这份唯一权威规则手册无法丢失。

内容

为何构建

同一所有者开发的多个大型产品应用与数十个解耦可复用子模块，不断重新推导出相同的来之不易的规则——并不断遭遇同一类失败：测试与状态报告宣称成功，而功能对终端用户却已失效（即“伪成功”与“伪失败”）。Constitution中的每一条法医锚点均记录了一起真实事件（如2026年5月20日D3音频路由“伪成功”案例，验证通过时“正在使用的编解码器”字段为空；或2026年6月25日巨型按钮UI案例，令牌相等性测试通过，实际界面却已损坏）。Constitution的存在，正是为了从机制上彻底杜绝这类虚假成功，且一劳永逸、普适全局——使规则不因项目而异，亦不致被悄然遗忘。

为何是颠覆性变革

它将工程文化从“文档期望遵循”转变为“继承、版本化、机械强制的法律”——其差异犹如风格指南与编译器之别。单个子模块升级即可同步更新整个系统的规则，原子化且可追溯。一条反伪成功契约在每个消费仓库中的存在，并非依赖信任，而是通过构造确保：传播门会在全系统范围内逐字检索条款编号，配套的变异测试则证明该门本身并非虚设——甚至连强制执行机制都受到强制。治理不再是无人问津的维基愿景，而成为可审计、可测试的事实，可直接交由CI任务验证。

创新之处

- **Constitution**作为子模块——工程法规如代码般分发与版本锁定，采用精心设计的v1.0.0式标签与项目级锁定，确保每个仓库明确知晓其所遵循的法规版本。
- 反伪成功作为一级法医原则——每条条款均可追溯至操作员的原话指令，且常对应具体的真实事件，使规则手册如判例法般呈现，而非主观意见。
- 规则元测试（§1.1）——每个门均配有一项必须触发PASS→FAIL转换的变异测试，因此“门非虚设”并非断言，而是每次运行均得以验证；一个永不失败的门，其危害甚于无门。
- 赢得的普适性——明确的四步测试决定规则是否真正普适或仅限项目特定，确保基础规则精简、可移植，且免受供应商污染。

跨产品使用方式（赋予的能力）

作为强制治理支柱，HelixConstitution并非家族可参考的文档——它是家族赖以构建的承重结构：

- 治理骨架：每个Helix/vasic-digital项目均将其作为子模块引入，并从CLAUDE.md/AGENTS.md/QWEN.md或自身的Constitution.md导入；规则自首次提交起即无条件适用，无项目可选择退出。
- 门与强制条款：定义了四层覆盖模型——源码存在、构建通过、运行时行为、门非虚设——功能需在四个层级均通过方可视为完成，此外还包含不断扩充的强制条款清单：凭证处理（§11.4.10）、文档同步（§11.4.60）、容器子模块强制（§11.4.76）、CodeGraph（§11.4.78）、强制测试类型覆盖（§11.4.169）等。
- 传播机制：CM-COVENANT-114-NNN-PROPAGATION门确保契约原文在所有消费仓库中存在，契约无法在系统某个角落被悄然移除；不合规将成为硬性发布阻断，无任何绕过标志可供规避。
- 发现机制：submodules-catalogue.md将“我们是否已有实现X功能的模块？”转化为一目了然的答案，在新模块搭建前即消除重复工作。
- AI代理的一致性：同一法规以完全相同的形式表达于每个CLI代理（Claude Code、Codex/Cursor/Aider/OpenCode/Crush/Kimi等通过AGENTS.md，Qwen Code通过QWEN.md），无论哪种工具接触代码，均遵循同一契约。

内容

最大技术挑战及解决方案

- 从任意嵌套深度定位子模块 —— 即使规则隐藏在三层子模块之下，也必须能找到其所在的法典，而无需预知其位置 → find_constitution.sh 脚本向上遍历父目录，递归跟踪 Git 超级项目指针，支持 CONSTITUTION_DIR 覆盖设置及两种布局（constitution/、submodules/constitution/），确保无论嵌套多深，解析过程均可确定性完成。
- 在四家 Git 托管平台上维护单一权威仓库 —— 若镜像与主仓库出现偏差，便毫无价值 → install_upstreams.sh 读取声明式的 Upstreams/*.sh 远程配置，为 origin 设置多个推送 URL，使单次 git push 能原子性地同步至 GitHub（主仓库）、GitLab、GitFlic 及 GitVerse，确保任何镜像均不会滞后。
- 防止规则膨胀 / 通用基础层被项目污染 —— 每一次“直接加在这里”的诱惑都会削弱可移植性 → 通过“赢得普适性”四步测试及 §11.4.17 条款的“通用 vs 项目”分类机制，对每一条新规则进行审查，将项目特定的内容强制下沉至项目层，确保其归属得当。
- 验证继承门控机制的有效性 —— 从未失效的门控无法取信于人 → meta_test_inheritance.sh 这一哨兵级元测试会故意删除 §11.4 锚点，并断言门控能成功拦截，从而持续验证强制机制本身是否存在隐性失效。

技术栈

- **Git 子模块继承** —— 原因：Git 子模块是唯一能让规则手册既保持权威性，又能按消费者需求进行版本锁定的机制，升级过程需通过显式、可审查的版本提升而非静默复制粘贴完成；实现：消费项目添加子模块并 @import 其代理文件，三层架构自顶向下执行，每个边界均严格遵循“不削弱仅扩展”的契约。
- **find_constitution.sh** —— 原因：若深度嵌套的代码无法可靠找到规则，规则便形同虚设，而硬编码路径一旦项目重组便会失效；实现：通过父目录遍历及 git rev-parse --show-superproject-working-tree 递归调用，辅以 CONSTITUTION_DIR 覆盖设置，支持两种布局的解析。
- **install_upstreams.sh + Upstreams/** —— 原因：四平台冗余只有在维护零额外成本时才真正有效，否则镜像终将腐化；实现：声明式的远程配置 .sh 文件被整合为单一多 URL 的 origin，将四次推送压缩为一次操作。
- **§1.1 变异元测试** —— 原因：从不失效的门控比没有门控更危险，因其会制造虚假的安全感；实现：每个门控均配对一个通过 sed 删除或重命名的变异测试，必须触发从 PASS 到 FAIL 的转变，随后恢复原状，确保每次运行时门控均能真正生效。
- **传播门控 (CM-COVENANT-114-NNN-PROPAGATION)** —— 原因：契约只有在所有消费者中均可验证存在时才具备普适性，而非仅限于旗舰仓库；实现：通过跨消费者的条款编号字面匹配检查，并辅以 §1.1 变异测试验证传播检查本身是否可失效。
- **submodules-catalogue.md (§11.4.74)** —— 原因：违反“禁止重复”纪律的最快速径，便是不知道自己已拥有什么；实现：一个包含 142 个仓库、按能力分组的清单，在搭建任何新内容前，均需在追踪系统中记录目录检查结果。
- **多格式导出** —— 原因：同一法典必须同时满足人类阅读、工具解析及归档保存的需求；实现：每份规范文档均从同一源生成 .md / .html / .pdf / .docx 格式。

内容

状态与诚信说明

- **状态**：已发布。当前作为子模块在整个系统中（公共官方及镜像仓库）积极维护与版本更新。
- **许可证**：待定——源材料中未明确说明；发布前请核对仓库 LICENSE 文件。
- **其他上游镜像**：GitLab helixdevelopment1/helixconstitution、GitFlic helixdevelopment/helixconstitution、GitVerse helixdevelopment/HelixConstitution。

优先级别：Helix-核心——Helix系列构建的强制性治理基石。